

組合語言與嵌入式系統Midterm Project

113學年度第1學期

老師：朱守禮 老師

班別：資工二甲

組別：第二組

學生：

11227203 謝采凌

11227236 曹湘婷

11227246 徐雨瑄

一、背景

我們的主要目標是透過 ARM 組合語言程式設計，學習如何利用工具（如 GCC、GAS、GDB 和 Code::Blocks）來撰寫、調試並驗證程式。不僅加深了我們對 ARM 處理器架構的理解，還讓我們實際操作了嵌入式系統開發環境。核心任務是撰寫兩個主要函數（NAME 和 ID）。

為三個主要部分：

1. NAME 函數：負責顯示組別名稱與組員姓名。
2. ID 函數：負責接收並顯示組員的學號，並計算學號的總和。
3. MAIN 函數：整合上述兩個函數，並顯示完整的組別資料與學號總和。

二、方法

使用的工具

- Code::Blocks：用於編寫、編譯與調試 ARM 組合語言程式，為開發過程提供了圖形化界面。
- Notepad++：用來編輯組合語言檔案，便於程式的撰寫與修改。
- WinSCP：用來將編寫好的程式從本地電腦上傳至虛擬機，或從虛擬機下載檔案進行修改，確保能夠順利在 ARM 虛擬機環境中運行和測試程式。

程式重點說明

本次程式分為三個函數檔：name.s、id.s 與 main.s。

- name.s：將組別名稱 teamNo 以及三位組員的英文姓名 name1、name2、name3 以 .asciz 的方式寫在 .data 段，並設為全域變數。在程式進入時，以 stmfd sp!, {lr} 保存返回位址，接著依序將字串的位址載入 r0，並使用多次 bl printf 輸出組別名稱與每位組員姓名。

所有輸出動作都在 name 函式內完成。最後再以 ldmfd sp!, {lr} 還原 LR，並以 mov pc, lr 正確返回 main，確保函式呼叫流程正常。整個 name.s 的功能為單純輸出固定資訊，不包含運算或條件判斷。

- id.s：在 .data 段中宣告 id1、id2、id3 與 idSum 四個 .word 變數，並設為全域，使 main.s 能在之後直接使用讀入的學號與加總結果。在執行 rsbs 前先用 mov r4, sp 與 mov r5, lr 備份原本的 SP 與 LR，再在 rsbs 後立即寫回，使 stack 與 LR 不被破壞。

學號輸入部分，我使用三次 scanf("%d")，依序讀入三位組員的學號並寫入對應的 id1、id2 與 id3。最終將運算出的結果寫入 idSum。接著再用 scanf("%s") 讀取命令字元 command。若輸入為 'p'，就印出三組學號及加總結果；若不是 'p' 則直接跳回 main，不輸出任何資料。

- main.s：先以 bl name 呼叫 name.s 輸出組別名稱與三位組員姓名，接著再以 bl id 呼叫 id.s 進行輸入與加總處理。使用 printf 將所有資訊整合輸出一次。當所有內容完成輸出後 main.s 即結束程式。

三、結果

程式順利執行並達成預期的功能。當使用者運行程式時，程式會先顯示組別名稱與三位組員的姓名，接著提示使用者輸入每位組員的學號，並計算學號總和。最終，程式會輸出每位組員的學號以及總和，並顯示在螢幕上。以下是程式執行的輸出結果：

```
*****Print Name*****
Team 2
Hsieh Tsai-Ling
Cao Xiang-Ting
Xu Yu-Xuan
*****End Print*****

Function2: ID:
*****Input ID*****
** Please Enter Member 1 ID : **
11227203
** Please Enter Member 2 ID : **
11227236
** Please Enter Member 3 ID : **
11227246
** Please Enter Command **
P
*****Print Team Member ID and ID Summation*****
11227203
11227236
11227246

ID Summation = 33681685
*****End Print*****

Main Function:
*****Print All*****
11227203 Hsieh Tsai-Ling
11227236 Cao Xiang-Ting
11227246 Xu Yu-Xuan
ID Summation = 33681685
*****End Print*****

Process returned 20 (0x14)   execution time : 14.048 s
Press ENTER to continue.
█
```

CPU Registers		
Regis	Hex	Integer
r0	0x0	0
r1	0xab506e	11227246
r2	0xab5064	11227236
r3	0x201f115	33681685
r4	0xbefff4c0	3204445376
r5	0x109ca	68042
r6	0x83a4	33700
r7	0x0	0
r8	0x0	0
r9	0x0	0
r10	0x40026000	1073897472
r11	0x0	0

入值成功由 scanf 寫入記憶體，再由程式邏輯讀入至 CPU 暫存器運算

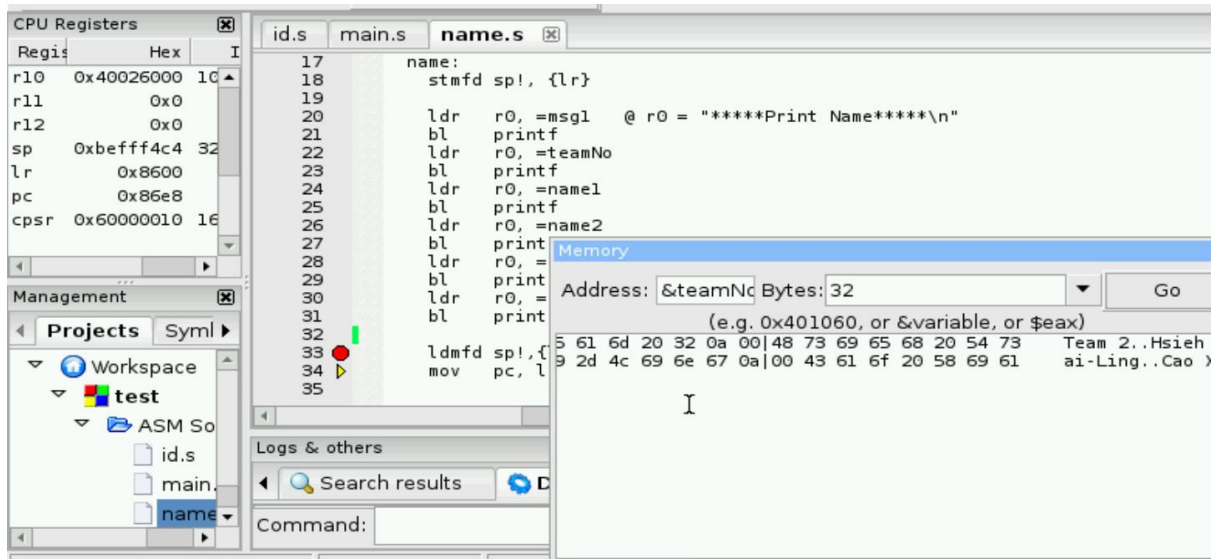
Disassembly		
Function:		
Frame start: 0xbefff4c4		
0x8470	ldr	r0, [pc, #304] ; 0x85a8 <Co
0x8474	bl	0x8398 <printf>
0x8478	ldr	r0, [pc, #300] ; 0x85ac <Co
0x847c	bl	0x8398 <printf>
0x8480	ldr	r0, [pc, #296] ; 0x85b0 <Co
0x8484	ldr	r1, [pc, #296] ; 0x85b4 <Co
0x8488	bl	0x838c <scanf>
0x848c	ldr	r0, [pc, #292] ; 0x85b8 <Co
0x8490	bl	0x8398 <printf>
0x8494	ldr	r0, [pc, #276] ; 0x85b0 <Co
0x8498	ldr	r1, [pc, #284] ; 0x85bc <Co
0x849c	bl	0x838c <scanf>
0x84a0	ldr	r0, [pc, #280] ; 0x85c0 <Co
0x84a4	bl	0x8398 <printf>
0x84a8	ldr	r0, [pc, #256] ; 0x85b0 <Co

ldr r0, [pc, #304] → 將字串常數的位址載入 r0 (準備給 printf)

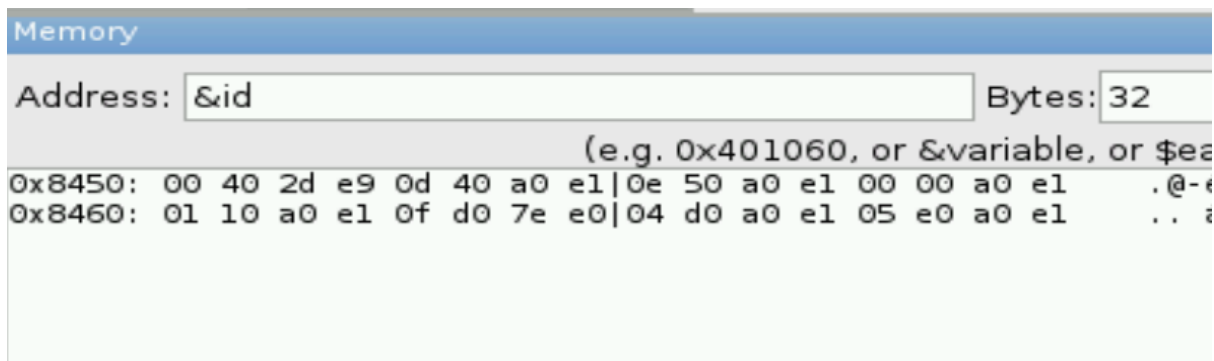
bl printf → 呼叫 printf 顯示文字 (例如 “Please Enter Member 1 ID”)

ldr r1, [...] → 載入 id1/id2/id3 的記憶體位址 (準備傳給 scanf)

bl scanf → 執行輸入 (把學號寫入記憶體)



內容	ASCII / Hex	說明
"Team 2\n"	54 65 61 6d 20 32 0a	組別名稱
"Hsieh Tsai-Ling\n"	48 73 69 65 68 20 ...	組員 1
"Cao Xiang-Ting\n"	43 61 6f 20 58 69 ...	組員 2
"Xu Yu-Xuan\n"	58 75 20 59 75 2d ...	組員 3



```

Memory
Address: &id1 Bytes: 32
(e.g. 0x401060, or &variable, or $eax)
0x109be: 43 50 ab 00 64 50 ab 00|6e 50 ab 00 15 f1 01 02 CP
0x109ce: 00 00 00 00 00 00 00 00|00 00 00 00 00 00 00 00 ..

```

```

Memory
Address: &id2 Bytes: 32
(e.g. 0x401060, or &variable, or $eax)
0x109c2: 64 50 ab 00 6e 50 ab 00|15 f1 01 02 00 00 00 00 dP«.nP«
0x109d2: 00 00 00 00 00 00 00 00|00 00 00 00 00 00 00 00 .....

```

```

Memory
Address: &id3 Bytes: 32
(e.g. 0x401060, or &variable, or $eax)
0x109c6: 6e 50 ab 00 15 f1 01 02|00 00 00 00 00 00 00 00 nP«.nP«
0x109d6: 00 00 00 00 00 00 00 00|00 00 00 00 00 00 00 00 .....

```

```

Memory
Address: &idSum Bytes: 32
(e.g. 0x401060, or &variable, or $eax)
1 01 02 00 00 00 00 00|00 00 00 00 00 00 00 00 .ñ.....
0 00 00 00 00 00 00 00|00 00 00 00 00 00 00 00 .....

```

label	内容 (十進位)	起始位址	結束位址
id1	11227203	0x109b6	0x109b9
id2	11227236	0x109ba	0x109bd
id3	11227246	0x109be	0x109c1
idSum	33681685	0x109c2	0x109c5

指定項目說明

(1) 非 branch 的 Conditional Execution (至少 2 條)

- `addeq r3, r1, r2` @當 `id1 == id2` 時執行加法 (EQ) 。
- `addge r3, r3, #1` @當結果 ≥ 0 時 (GE) 執行遞增 。
- `ldrne r4, [r4, #1]` @當 `Z = 0` 時 (NE) 才會 Load 。

(2) Operand2 至少三種型式

立即數 (Immediate) :

- `addge r3, r3, #1`
- `mov r0, #0`

暫存器 (Register) :

- `sub r3, r1`
- `cmp r0, r1`

暫存器加移位 (Register + Shift) :

- `add r3, r3, r1, ror #31`

(3) 三種以上 Load/Store addressing mode

程式中使用到多種 addressing mode · 包括 :

- `ldr r1, [r1]`
- `ldr r4, [r4, #1]`
- `ldr r6, [r4, r5]`

● `ldr r6, [r4, #3]!`

四、討論

在一開始，我們在寫 `.s` 檔案時，經常出現記憶體位置錯誤的情況。由於 ARM 組合語言對記憶體管理的要求非常嚴格，稍微疏忽就可能導致變數或資料的記憶體位置錯亂。例如，在分配資料區段時，我們未能正確設置起始位址或使用了錯誤的載入方式，這使得程式執行時無法正確存取資料，導致程式異常終止或無法按預期運行。

此外，我們在編寫程式碼時，格式常常出錯，尤其是對齊和標籤的使用上。例如，有時標籤的定義不清楚或指令的對齊錯誤，造成編譯器無法正確識別函數或變數。這類格式錯誤頻繁出現，尤其是在函數之間的跳轉和堆疊操作部分，導致我們不得不反覆檢查每一行程式碼。更糟糕的是，我們的 `main` 函數無法找到入口點，這通常是因為函數標籤或外部符號設定錯誤。在 ARM 組合語言中，函數標籤是很重要的，它們必須與連結器中的符號對應，而如果 `main` 函數的入口標籤設置錯誤，程式便無法正確啟動或執行。這個問題讓我們多次嘗試後仍無法順利開始程式的執行。

在最後程式中的迴圈也經常卡住。這通常是因為迴圈條件設置錯誤，或者在迴圈內部的邏輯錯誤。例如，在學號輸入的部分，程式無法正確進入迴圈，或者進入後無法按預期跳出，導致程式無法繼續執行，甚至進入無窮迴圈。面對這些問題，我們進行了多次調整與測試，逐漸理解了 ARM 組合語言中堆疊操作、記憶體管理和函數呼叫的細節，也學會了如何處理常見的格式錯誤和邏輯問題。雖然這些問題帶來了不少挑戰，但也讓我們在實際開發過程中獲得了寶貴的經驗。

五、心得感想

這次學習讓我們更加深入了解 ARM 組合語言的基礎運作，並實際操作了嵌入式系統開發過程中的各種工具。學會了如何使用 GCC、GAS 進行編譯，並藉由 GDB 進行除錯。對於未來的開發，我們將更加注重組合語言程式中的記憶體管理和格式檢查。首先，我們會更仔細地設計記憶體區塊，確保每個變數和資料結構的儲存位置正確無誤。其次，為了避免格式錯誤，我們會在寫程式之前深入理解每條指令的語法與格

式，並在開發過程中多進行測試與調試，確保每段程式碼都能順利執行。對於迴圈和條件判斷的部分，我們將加強邏輯檢查，避免無窮迴圈和錯誤的跳出條件。

六、分工

成員 1：謝采凌 — name.s 與資料區設計

負責撰寫 name.s 程式，設計並定義組別名稱與成員姓名等 .data 資料區段，建立資料的儲存格式與記憶體配置。負責測試姓名資料是否能正確讀取、對齊與顯示，確保組別與成員基本資訊能準確輸出。

成員 2：曹湘婷 — id.s 與學號輸入處理

撰寫 id.s 程式，負責讀取、儲存並處理每位組員的學號，包含輸入流程、學號的記憶體存放設計，以及學號總和的計算。進行學號格式、輸入正確性與加總結果的測試，確保數據處理無誤。

成員 3：徐雨瑄 — main.s 整合與流程控制

撰寫 main.s 並整合 name.s 與 id.s，負責控制程式主流程，依序執行成員姓名顯示與學號處理功能。測試整體執行順序、流程連結與程式在 ARM 環境中的運作狀態，確保整支程式能順利啟動、執行與結束。